Lab 5: Static Code Analysis

Submission Report

Name	Mohammed Shahid Javed Mokashi
SRN	PES1UG24AM378
GitHub Repository	https://github.com/PES1UG24AM378/L5_17_society_dues
Assigned file	society_dues.py (Apartment Society Maintenance Dues Ledger)
Tools used	Pylint, Bandit, Flake8

1. Results Summary

Tool	Before	After
Pylint	5.81 / 10	10.00 / 10
Bandit	3 issues (0 High, 2 Medium, 1 Low)	0 issues
Flake8	5 issues	0 issues

All Bandit issues handled, Pylint above 9.5 and all Flake8 formatting issues fixed, as required. Program behaviour is unchanged (output: 2500, 100, ['A-102']).

2. Issues Identified and Fixed

Issue	Type	Line(s)	Description	Fix Approach
SQL injection (B608)	Security (Medium)	9	addDue built the INSERT with an f-string; input like x'); DROP TABLE dues;-- would run as SQL.	Parameterized query: INSERT INTO dues VALUES (?, ?, ?, 0) with a values tuple.
Use of eval (B307 / W0123)	Security (Medium)	21	penalty(rule) ran eval() on a caller-supplied string, allowing arbitrary code execution.	Replaced with an ast-based evaluator (_safe_eval) allowing only numbers and + - * /.
Hardcoded password (B105)	Security (Low)	5	TREASURER_PASSWORD = "flat#101" committed in source.	Read from the TREASURER_PASSWORD environment variable via os.environ.get.
Mutable default argument (W0102)	Bug	8	history=[] is created once and shared across calls, so history leaked between calls.	Default set to None; a new list is created inside the function.
Broad exception (W0718)	Quality	22	except Exception swallowed every error and hid real bugs.	Catch only ValueError, SyntaxError, ZeroDivisionError.
Inconsistent returns (R1710)	Bug	12-16	collect returned an amount when found but implicitly None otherwise.	Added explicit return 0 when no matching due exists.
Naming (C0103)	Style	8	addDue is not snake_case.	Renamed to add_due and updated the caller.
Missing docstrings (C0114/C0116)	Quality	1, 8, 12, 19, 27, 34	Module and all functions were undocumented.	Added docstrings to the module and every function.
Singleton comparison (E711 / C0121)	Style	29	rows != None	Changed to rows is not None.
Line length (E501 / C0301)	Style	4, 13, 15	Lines longer than 79 characters.	Wrapped long SQL strings and calls across lines.
Missing blank lines (E302)	Style	12	No blank lines before def collect.	Two blank lines between top-level functions.
No main guard	Quality	42	main() ran on import.	Wrapped in if __name__ == "__main__":

3. Git Commits

```
e50a843 Fix Bandit issues: remove hardcoded password, parameterize SQL, replace eval with safe AST
    evaluator
0d104c9 Fix Pylint issues: docstrings, snake_case, mutable default, consistent returns, narrow
    except, main guard
b27e55f Fix Flake8 issues: line length, blank lines, None comparison
ead9b5d Add before/after reports, issues.md and reflection.md
05c7df9 Add combined submission report
```

Commits 1-3 fix the Bandit, Pylint and Flake8 issues respectively; the rest add the reports and documents.

4. Final Code: society_dues.py

```
"""Apartment society maintenance dues ledger."""
import ast
import operator
import os
import sqlite3

conn = sqlite3.connect(":memory:")
conn.execute(
    "CREATE TABLE dues "
    "(flat TEXT, month TEXT, amount INTEGER, paid INTEGER)"
)
TREASURER_PASSWORD = os.environ.get("TREASURER_PASSWORD")


def add_due(flat, month, amount, history=None):
    """Bill a flat for a month and record it in the history list."""
    if history is None:
        history = []
    conn.execute(
        "INSERT INTO dues VALUES (?, ?, ?, 0)", (flat, month, amount)
    )
    history.append((flat, month))
    return history


def collect(flat, month):
    """Mark a due as paid and return the amount, or 0 if no due exists."""
    row = conn.execute(
        "SELECT amount FROM dues WHERE flat = ? AND month = ?",
        (flat, month),
    ).fetchone()
    if row:
        conn.execute(
            "UPDATE dues SET paid = 1 WHERE flat = ? AND month = ?",
            (flat, month),
        )
        return row[0]
    return 0


_OPS = {
    ast.Add: operator.add,
    ast.Sub: operator.sub,
    ast.Mult: operator.mul,
    ast.Div: operator.truediv,
}


def _safe_eval(node):
    """Evaluate a numeric arithmetic AST node without using eval()."""
    if isinstance(node, ast.Expression):
        return _safe_eval(node.body)
    if isinstance(node, ast.Constant) and isinstance(
        node.value, (int, float)
    ):
        return node.value
    if isinstance(node, ast.BinOp) and type(node.op) in _OPS:
        return _OPS[type(node.op)](
            _safe_eval(node.left), _safe_eval(node.right)
        )
    raise ValueError("unsupported expression")


def penalty(rule):
    """Return the late fee from an arithmetic rule string, 0 if invalid."""
    try:
        late_fee = _safe_eval(ast.parse(rule, mode="eval"))
    except (ValueError, SyntaxError, ZeroDivisionError):
        late_fee = 0
    return late_fee


def pending():
    """Return the flats that still have unpaid dues."""
    rows = conn.execute("SELECT flat FROM dues WHERE paid = 0").fetchall()
    if rows is not None:
        return [r[0] for r in rows]
    return []
```

```
def main():
    """Run a short demonstration of the ledger."""
    add_due("A-101", "Oct", 2500)
    add_due("A-102", "Oct", 2500)
    print(collect("A-101", "Oct"))
    print(penalty("50 * 2"))
    print(pending())


if __name__ == "__main__":
    main()
```

5. Static Analysis Reports

Pylint report - before

```
************* Module society_dues
society_dues.py:13:0: C0301: Line too long (104/100) (line-too-long)
society_dues.py:1:0: C0114: Missing module docstring (missing-module-docstring)
society_dues.py:8:0: C0116: Missing function or method docstring (missing-function-docstring)
society_dues.py:8:0: C0103: Function name "addDue" doesn't conform to snake_case naming style (invalid-
    name)
society_dues.py:8:0: W0102: Dangerous default value [] as argument (dangerous-default-value)
society_dues.py:12:0: C0116: Missing function or method docstring (missing-function-docstring)
society_dues.py:12:0: R1710: Either all return statements in a function should return an expression, or
    none of them should. (inconsistent-return-statements)
society_dues.py:19:0: C0116: Missing function or method docstring (missing-function-docstring)
society_dues.py:22:11: W0718: Catching too general exception Exception (broad-exception-caught)
society_dues.py:21:19: W0123: Use of eval (eval-used)
society_dues.py:27:0: C0116: Missing function or method docstring (missing-function-docstring)
society_dues.py:29:7: C0121: Comparison 'rows != None' should be 'rows is not None' (singleton-
    comparison)
society_dues.py:34:0: C0116: Missing function or method docstring (missing-function-docstring)

-----------------------------------
Your code has been rated at 5.81/10
```

Pylint report - after

```
-------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

Bandit report - before

```
Run started:2026-10-16 03:44:15.765426+00:00

Test results:
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'flat#101'
   Severity: Low   Confidence: Medium
   CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b105_hardcoded_password_string.html
   Location: ./society_dues.py:5:21
4       conn.execute("CREATE TABLE dues (flat TEXT, month TEXT, amount INTEGER, paid INTEGER)")
5       TREASURER_PASSWORD = "flat#101"
6

--------------------------------------------------
>> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based query
    construction.
   Severity: Medium   Confidence: Medium
   CWE: CWE-89 (https://cwe.mitre.org/data/definitions/89.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b608_hardcoded_sql_expressions.html
   Location: ./society_dues.py:9:19
8       def addDue(flat, month, amount, history=[]):
9           conn.execute(f"INSERT INTO dues VALUES ('{flat}', '{month}', {amount}, 0)")
10          history.append((flat, month))

--------------------------------------------------
>> Issue: [B307:blacklist] Use of possibly insecure function - consider using safer ast.literal_eval.
   Severity: Medium   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b307-eval
   Location: ./society_dues.py:21:19
20          try:
21              late_fee = eval(rule)
22          except Exception:

--------------------------------------------------

Code scanned:
        Total lines of code: 31
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 1
                Medium: 2
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 2
                High: 1
Files skipped (0):
```

Bandit report - after

```
Run started:2026-10-16 03:45:03.770296+00:00

Test results:
        No issues identified.

Code scanned:
        Total lines of code: 74
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
Files skipped (0):
```

Flake8 report - before

```
society_dues.py:4:80: E501 line too long (87 > 79 characters)
society_dues.py:12:1: E302 expected 2 blank lines, found 0
society_dues.py:13:80: E501 line too long (104 > 79 characters)
society_dues.py:15:80: E501 line too long (92 > 79 characters)
society_dues.py:29:13: E711 comparison to None should be 'if cond is not None:'
```

Flake8 report - after

```
(no issues reported)
```

6. Reflection

1. Which issues were the easiest and hardest to fix?

Easiest were the style issues (Flake8 line length, blank lines, `is not None`, docstrings, snake_case) because the tools say exactly what to change. Hardest was replacing `eval()`: simply deleting it would break `penalty("50 * 2")`, so I had to write a safe AST-based evaluator that keeps the same behaviour. The SQL injection fix was also easy to apply, but it needed understanding why parameterized queries are safe.

2. Did the tools report any false positives?

Mostly no. The hardcoded password warning (B105) was low severity and the password was only a demo value, but it was still a real bad practice, so I fixed it by moving it to an environment variable. Pylint's missing-docstring messages are style rules rather than bugs, so they are "noise" in a small script but harmless to fix.

3. How would you integrate static analysis into your workflow?

Run Flake8 and Pylint locally (editor plugin or pre-commit hook) so problems are caught before committing, and run all three tools (Pylint, Flake8, Bandit) in a CI pipeline such as GitHub Actions that fails the pull request if Bandit finds anything or the Pylint score drops below a threshold like 9.5.

4. What tangible improvements did you see in code quality, readability and robustness?

Pylint went from 5.81 to 10.00, Bandit from 3 issues to 0 and Flake8 from 5 to 0. The code is safer (no injection or code execution), more robust (no shared mutable state, no swallowed exceptions, consistent return values) and more readable (docstrings, consistent naming and formatting). Behaviour is unchanged: the program still prints `2500`, `100`, `['A-102']`.

7. Final Program Output

```
2500
100
['A-102']
```